CIP TEAM HARDWARE GUIDE

WeCare
Health, Equality and Technology

Optional IoT Health Monitoring Modules

ALL MODULES OPTIONAL

Optional health hub prototype. Option A (ESP32, Arduino C++, includes ESP32-CAM) or Option B (Raspberry Pi, Python, full beginner walkthrough). All I2C sensors share one bus. These are for educational prototyping only, NOT medical-grade.

MAX30102 HR/SpO2 | MLX90614 IR Temp | BME680 Air Quality | PIR Motion | Sound Sensor | ESP32-CAM

Sensor Overview

All modules optional . choose your board path

HEALTH DISCLAIMER

These sensors are for EDUCATIONAL PROTOTYPING ONLY. NOT medical-grade. NOT for clinical diagnosis.

I2C BUS SHARING

MAX30102 (0x57), MLX90614 (0x5A), BME680 (0x76) all share the same two wires (SDA + SCL). Each has a unique address so no conflicts.

MAX30102 Heart Rate + SpO2 (I2C 0x57)

Optical fingertip sensor. Red+IR LEDs. Heart rate BPM and blood oxygen %. Place finger flat, do not press hard.

MLX90614 IR Body Temperature (I2C 0x5A)

Non-contact infrared thermometer. Point at forehead from 2-3 cm. Returns ambient + body temperature.

BME680 Air Quality (I2C 0x76)

4-in-1: temp, humidity, pressure, VOC gas. Higher gas resistance = cleaner air. Needs 5-30 min warmup.

PIR Motion Sensor (Digital)

Detects human movement. Counts clinic visitors. Has sensitivity and delay pots. ESP32: GPIO13. Pi: GPIO17.

Sound Sensor (Analog)

Microphone module for noise monitoring. Needs ADS1115 on Pi. ESP32: GPIO35. Pi: ADS1115 AIN0.

ESP32-CAM (Option A only)

Camera + WiFi. MJPEG streaming. Needs FTDI programmer. NOT available with Raspberry Pi option.

A Option A: ESP32 as Development Board

Arduino C++ . built-in WiFi . ADC for sound . ESP32-CAM support

1. **Install Arduino IDE + ESP32 board**
   Add URL: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

2. **Install libraries**
   SparkFun MAX3010x, Adafruit MLX90614, Adafruit BME680 from Library Manager.

3. **Wire I2C sensors**
   All three share GPIO21 (SDA) + GPIO22 (SCL). PIR on GPIO13. Sound on GPIO35 (ADC).

4. **Test each sensor individually**
   Upload a test sketch for each sensor one at a time.

5. **ESP32-CAM**
   Separate board. Wire to FTDI. IO0 to GND for upload. Flash CameraWebServer example.

Option A: All Sensors to ESP32

Sensor pin	Board pin	Wire colour	Notes
I2C SDA (all 3 sensors)	**GPIO21**	#1E88E5	MAX30102+MLX90614+BME680
I2C SCL (all 3 sensors)	**GPIO22**	#42A5F5	Shared clock
MAX30102 INT	**GPIO2 (optional)**	#FF9800	Heartbeat interrupt
PIR OUT	**GPIO13**	#43A047	Motion detect
Sound AOUT	**GPIO35 (ADC)**	#FF9800	Analog noise

ARDUINO C++

```
// WeCare ESP32: all health hub sensors
#include <Wire.h>
#include "MAX30105.h"
#include <Adafruit_MLX90614.h>
#include <Adafruit_BME680.h>

MAX30105 hrSensor;
Adafruit_MLX90614 mlx;
Adafruit_BME680 bme;
#define PIR_PIN 13
#define SOUND_PIN 35
int visits = 0;
bool lastPir = false;

void setup() {
    Serial.begin(115200);
    hrSensor.begin(Wire, I2C_SPEED_FAST);
    hrSensor.setup();
    mlx.begin();
    bme.begin(0x76);
```

ARDUINO C++ CONT.

```
    bme.setTemperatureOversampling(BME680_OS_8X);
    bme.setHumidityOversampling(BME680_OS_2X);
    bme.setGasHeater(320, 150);
    pinMode(PIR_PIN, INPUT);
    Serial.println("WeCare Hub ready!");
}

void loop() {
    long ir = hrSensor.getIR();
    if (ir > 50000) Serial.printf("HR IR:%ld\n", ir);
    else Serial.println("No finger");

    Serial.printf("Body:%.1fC Ambient:%.1fC\n",
        mlx.readObjectTempC(), mlx.readAmbientTempC());

    if (bme.performReading())
        Serial.printf("AQ:%.1fC %.1f%% %.0fKOhm\n",
            bme.temperature, bme.humidity, bme.gas_resistance/1000.0);

    bool pir = digitalRead(PIR_PIN);
    if (pir && !lastPir) { visits++; Serial.printf("Visitor #%d\n", visits); }
    lastPir = pir;

    Serial.printf("Noise:%d\n---\n", analogRead(SOUND_PIN));
    delay(2000);
}
```

B Option B: Raspberry Pi as Development Board

Python 3 . full beginner walkthrough . every step from zero . no ESP32-CAM

WHAT YOU NEED

Raspberry Pi 3B+/4/5. Micro SD 16GB+. Power cable. Laptop with WiFi. SD card reader. ADS1115 ADC board (for sound sensor). Jumper wires. The I2C sensors.

NO ESP32-CAM ON PI

The ESP32-CAM is a standalone board only available with Option A. For camera on Pi, use a Raspberry Pi Camera Module (not covered here).

Phase 1: Download and Flash the Operating System

1 Download Raspberry Pi Imager on your laptop

Open your browser (Chrome, Firefox, Edge). Go to raspberrypi.com/software and click the download button for your operating system. Windows: download the .exe file. Mac: download the .dmg file. Linux: download the .deb file. Run the installer and follow the instructions.

2 Insert your SD card into your laptop

Take the micro SD card (16 GB or more) and put it into your SD card reader. If your laptop has a built-in SD slot, insert it directly. If not, plug in a USB SD card reader adapter. Your computer should detect it automatically. You do not need to format it.

3 Open Imager and choose the operating system

Open Raspberry Pi Imager. Click the CHOOSE OS button. Scroll down and click Raspberry Pi OS (other). Then click Raspberry Pi OS Lite (64-bit). This is a small version with no desktop, which is perfect for sensor work.

4 Configure WiFi and SSH before writing (very important!)

Click the gear icon in the bottom-right corner of Imager (or press Ctrl+Shift+X). This opens the settings panel. Turn ON 'Set hostname' and type: raspberrypi. Turn ON 'Enable SSH' and select 'Use password authentication'. Set the username to: pi and choose a password you will remember. Turn ON 'Configure wireless LAN' and type your WiFi network name and password exactly. Set the country to your WiFi country (DE for Germany). Click SAVE.

5 Write the image to the SD card

Click CHOOSE STORAGE and select your SD card. Then click WRITE. The imager will download the operating system and write it to the card. This takes 3 to 8 minutes. When it says Write Successful, click Continue and remove the SD card from your laptop.

6 Put the SD card into the Pi and power it on

Slide the SD card into the slot on the bottom of the Raspberry Pi. The metal contacts face up toward the board. Connect the power cable. For Pi 3+: plug in the micro USB cable. For Pi 5: plug in the USB-C cable. The red LED turns on (power). The green LED flickers (booting). Wait 90 seconds for the first boot to finish.

Phase 2: Connect to Your Pi and Set It Up

7 Open a terminal on your laptop

On Windows: press the Windows key, type 'cmd', and press Enter. Or search for 'Windows Terminal' or 'PowerShell'. On Mac: press Cmd+Space, type 'Terminal', and press Enter. On Linux: press Ctrl+Alt+T.

8 Connect to the Pi via SSH

Type the command below and press Enter. The first time, it will ask 'Are you sure you want to continue connecting?' Type yes and press Enter. Then type your password and press Enter. You will NOT see the password as you type. That is normal. When you see pi@raspberrypi:~ $ you are connected!

```
ssh pi@raspberrypi.local
# If this does not work, find the Pi's IP from your router
# (usually at 192.168.1.1) and use:
# ssh pi@192.168.X.XXX
```

9 Update all software on the Pi

Type these two commands one at a time. The first updates the list of available software. The second installs all updates. Press Y and Enter if asked to confirm. This may take 5 to 10 minutes the first time.

```
sudo apt update
sudo apt upgrade -y
```

10 Enable the I2C interface

I2C is how the Pi talks to sensors like the ADS1115. It is turned off by default. Run the command below. A blue menu appears. Use the arrow keys to go to '3 Interface Options', press Enter. Select 'I5 I2C', press Enter. Select 'Yes', press Enter. Select 'Finish', press Enter. Then reboot and reconnect.

```
sudo raspi-config
# After enabling I2C, reboot:
sudo reboot
# Wait 30 seconds, then reconnect:
ssh pi@raspberrypi.local
```

Phase 3: Install Python Libraries

11 Install all the Python sensor libraries

Type each command below one at a time. Each one downloads a software package that lets Python talk to a specific sensor. The --break-system-packages flag is needed on newer Pi OS versions. Wait for each one to finish before typing the next. These three libraries are for the I2C health sensors.

```
pip3 install adafruit-blinka --break-system-packages
pip3 install adafruit-circuitpython-dht --break-system-packages
pip3 install adafruit-circuitpython-ads1x15 --break-system-packages
pip3 install RPi.GPIO --break-system-packages
pip3 install requests --break-system-packages
pip3 install adafruit-circuitpython-bme680 --break-system-packages
pip3 install adafruit-circuitpython-mlx90614 --break-system-packages
pip3 install max30102 --break-system-packages
```

12 Verify I2C sensors are detected

After wiring your I2C sensors (like the ADS1115), run this command. If wiring is correct, you will see numbers (like 48) in the grid. If you see only dashes, something is wrong with the wiring. Check SDA is on Pin 3, SCL is on Pin 5, and power/ground are connected.

```
i2cdetect -y 1
# Expected: 48 at address 0x48 (ADS1115)
```

Phase 4: Wire the WeCare Health Sensors

ALL I2C ON SAME BUS

Connect ALL sensor SDA wires together to GPIO2 (Pin 3). Connect ALL sensor SCL wires together to GPIO3 (Pin 5). Each sensor has a different address so they do not conflict.

Option B: All Sensors to Raspberry Pi

Sensor pin	Board pin	Wire colour	Notes
All I2C SDA	**GPIO2 (Pin 3)**	#1E88E5	MAX30102+MLX90614+BME680+ADS1115
All I2C SCL	**GPIO3 (Pin 5)**	#42A5F5	Shared I2C clock
All I2C VCC	**3.3V (Pin 1)**	#E53935	Power all sensors
All I2C GND	**GND (Pin 6)**	#424242	Ground all sensors
PIR OUT	**GPIO17 (Pin 11)**	#43A047	Motion detect
Sound AOUT	**ADS1115 AIN0**	#FF9800	Analog noise level

13 Verify all I2C sensors are detected

After wiring, run the I2C scan. You should see FOUR addresses: 48 (ADS1115), 57 (MAX30102), 5a (MLX90614), 76 (BME680). If any is missing, check that sensor's wiring.

```
i2cdetect -y 1
# Expected addresses: 48, 57, 5a, 76
```

Phase 5: Create and Run the Script

14 Create the script file

Open nano to create the Python file.

```
nano /home/pi/wecare_hub.py
```

15 Paste the code, save, and exit

Copy the full code block below, paste into nano, Ctrl+O Enter to save, Ctrl+X to exit.

PYTHON

```python
# /home/pi/wecare_hub.py - WeCare Health Hub complete script
import time, math, requests, board, busio
import adafruit_bme680
import adafruit_mlx90614
import adafruit_ads1x15.ads1115 as ADS
from adafruit_ads1x15.analog_in import AnalogIn
import RPi.GPIO as GPIO

# ====== REPLACE THESE ======
GATEWAY_IP = "192.168.X.X"  # WaziGate IP (or None to skip)
DEVICE_ID  = "wecare-hub"   # WaziGate device ID

# ====== I2C SENSORS (all share GPIO2/GPIO3) ======
i2c = busio.I2C(board.SCL, board.SDA)
bme = adafruit_bme680.Adafruit_BME680_I2C(i2c, address=0x76)
mlx = adafruit_mlx90614.MLX90614(i2c)
ads = ADS.ADS1115(i2c)
sound_ch = AnalogIn(ads, ADS.P0)

# ====== PIR MOTION SENSOR on GPIO17 ======
PIR_PIN = 17
visitor_count = 0
last_pir_state = False
GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN)

def send_to_wazigate(sensor_id, value):
    if not GATEWAY_IP or value is None:
        return
    try:
        url = f"http://{GATEWAY_IP}/api/v2/devices/{DEVICE_ID}/sensors/{sensor_id}/value"
        requests.post(url, json={"value": value}, timeout=5)
    except:
        pass

print("WeCare Health Hub started!")
print("BME680 gas sensor warming up (5-30 min for accurate VOC)...")

while True:
    print("--- Reading sensors ---")

    # Body temperature (MLX90614)
    try:
        body_temp = mlx.object_temperature
        ambient_temp = mlx.ambient_temperature
        print(f"Body temperature: {body_temp:.1f} C")
        print(f"Ambient: {ambient_temp:.1f} C")
        if body_temp > 37.5:
            print("  NOTE: Elevated temperature")
    except:
```

PYTHON CONT.

```
    body_temp = None
    print("MLX90614: read failed")

# Air quality (BME680)
try:
    aq_temp = bme.temperature
    aq_humidity = bme.humidity
    aq_gas = bme.gas / 1000.0  # KOhms
    print(f"Air temp: {aq_temp:.1f} C")
    print(f"Humidity: {aq_humidity:.1f} %")
    print(f"Gas (VOC): {aq_gas:.0f} KOhms")
    if aq_gas > 300:
        print("  Air quality: GOOD")
    elif aq_gas > 100:
        print("  Air quality: MODERATE")
    else:
        print("  Air quality: POOR")
except:
    aq_temp = aq_humidity = aq_gas = None
    print("BME680: read failed")

# Noise level (sound sensor via ADS1115)
try:
    noise_voltage = sound_ch.voltage
    if noise_voltage > 0.005:
        noise_db = 20 * math.log10(noise_voltage / 0.005)
    else:
        noise_db = 0
    print(f"Noise: ~{noise_db:.0f} dB")
except:
    noise_db = 0
    print("Sound sensor: read failed")

# Visitor counting (PIR)
pir_state = GPIO.input(PIR_PIN)
if pir_state and not last_pir_state:
    visitor_count += 1
    print(f"Motion! Visitor #{visitor_count}")
last_pir_state = pir_state
print(f"Total visitors today: {visitor_count}")

# Send all to WaziGate
print("--- Sending to WaziGate ---")
if body_temp is not None:
    send_to_wazigate("body_temp", round(body_temp, 1))
if aq_temp is not None:
    send_to_wazigate("aq_temp", round(aq_temp, 1))
if aq_humidity is not None:
    send_to_wazigate("aq_humidity", round(aq_humidity, 1))
if aq_gas is not None:
    send_to_wazigate("aq_gas", round(aq_gas, 0))
send_to_wazigate("visitors", visitor_count)
send_to_wazigate("noise_db", round(noise_db, 0))

print("--- Waiting 10 seconds ---")
time.sleep(10)
```

16 Test the script

Run the script. You should see body temperature, air quality, noise level, and visitor count. If a sensor fails, it prints 'read failed' but others keep working.

```
python3 /home/pi/wecare_hub.py
```

Phase 6: Auto-Start on Boot

17 Create and enable the systemd service

Makes the script start automatically on every boot.

```
sudo nano /etc/systemd/system/wecare.service
# Paste service config, change ExecStart to:
# /usr/bin/python3 /home/pi/wecare_hub.py
sudo systemctl daemon-reload
sudo systemctl enable wecare
sudo systemctl start wecare
sudo systemctl status wecare
```

WeCare Quick Reference

Sensor	Address / GPIO	Library	Install command	Unit
MAX30102	`I2C 0x57`	max30102 / SparkFu	`pip3 install max30102`	BPM/%
MLX90614	`I2C 0x5A`	adafruit-mlx90614	`pip3 install adafruit.`	C
BME680	`I2C 0x76`	adafruit-bme680	`pip3 install adafruit.`	C/%/KOhm
PIR	`ESP32 GPIO13 /`	GPIO	`pip3 install RPi.GPIO`	count
Sound	`ESP32 GPIO35 /`	ADC	`pip3 install adafruit.`	dB
ESP32-CAM	`Option A only`	esp32 Arduino	`Boards Manager`	MJPEG